

SPEC-1-NeuroPathway / Safe Space

Database Architecture and Child-Facing Pattern Recognition AI Reader - Code Method Extract

1. Method

The application will use PostgreSQL as the primary transactional database. PostgreSQL remains the authoritative system of record for regulated child records, safeguarding events, consent, tenant membership, AI outputs, adult review decisions, and audit trails.

The initial tenancy model will be a pooled multi-tenant PostgreSQL schema, where tenant-owned tables include a mandatory tenant_id column. For enterprise, NHS-adjacent, local authority, academy trust, or high-assurance safeguarding customers, the architecture must support promotion to a dedicated schema or dedicated database.

1.1 Core database pattern

- tenants
- users
- tenant_memberships
- roles
- permissions
- audit_logs
- children
- child_checkins
- consent_records
- safeguarding_cases
- safeguarding_events
- ai_outputs
- pattern_observations
- behaviour_patterns
- domain-specific tenant-owned tables with tenant_id
- shared reference tables without tenant_id where appropriate

1.2 Isolation strategy

- Application resolves active tenant from authenticated user/session.
- Database session receives the active tenant context.
- PostgreSQL Row-Level Security policies restrict reads/writes by tenant_id.
- All tenant-owned tables have indexes starting with tenant_id.
- Cross-tenant admin access uses separate privileged service roles with explicit audit logging.
- Child-facing AI workflows must never bypass tenant isolation or safeguarding policy checks.
- AI-derived records must be tenant-scoped and linked back to the source evidence used to generate them.

1.3 Tenancy model

```

Standard tenants
-> Shared PostgreSQL database
    -> Shared schema
        -> tenant_id scoped records

Enterprise / high-assurance tenants
-> Dedicated schema or dedicated database

Very high assurance tenants
-> Dedicated database
    -> Dedicated encryption boundary
        -> Region-specific deployment and backup policy

```

2. Child-Facing Pattern Recognition AI Reader

The child-facing side of NeuroPathway should include a Pattern Recognition AI Reader designed to help children understand their own routines, emotions, sensory experiences, communication preferences, and support needs.

This feature must be designed as a reflective wellbeing tool, not a diagnostic tool and not an automated safeguarding decision-maker.

2.1 Product framing

The child-facing UI should use gentle language such as My Patterns, My Day Map, What Helps Me, My Signals, or My Safe Space Reflections. Internally, the system may use technical terms such as `pattern_observations`, `behaviour_patterns`, `risk_classification`, and `safeguarding_events`.

```

Avoid child-facing labels:
- "You are dysregulated"
- "You are high risk"
- "You are deteriorating"
- "You have ADHD/autism/anxiety"

Use supportive reflections:
- "You often feel more settled after quiet time."
- "Busy places seem to make your day harder."
- "You seem to feel better when you know what is happening next."
- "You may want to ask a trusted adult for help with this."

```

2.2 Pattern reader pipeline

```

Child input
-> Safety pre-screen
-> Structured extraction
-> Pattern aggregation
-> Risk classification
-> Output policy decision
-> Child reflection or adult review queue
-> Audit event

```

2.3 Human-in-the-loop rule

AI finding	Child-facing behaviour	Adult/professional behaviour
Repeated tiredness	Gentle sleep/energy reflection	Optional parent/carer insight
Repeated sensory overload	Coping reflection	Share with approved adult if configured
Repeated sadness	Encourage trusted adult support	Create review item
Self-harm language	No generic chatbot response	Immediate safeguarding workflow
Disclosure/abuse indicator	Stop casual AI flow	Safeguarding escalation
Bullying pattern	Encourage trusted adult support	Review item for safeguarding role

3. SQL Code

3.1 child_checkins

```
create table child_record.child_checkins (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  child_id uuid not null references child_record.children(id),  
  checkin_type text not null,  
  mood_score int,  
  energy_score int,  
  sensory_score int,  
  free_text text,  
  created_at timestamptz not null default now(),  
  visibility text not null default 'child_and_approved_adults',  
  sensitivity_tier int not null default 3  
);  
  
create index child_checkins_tenant_child_time_idx  
on child_record.child_checkins (tenant_id, child_id, created_at desc);
```

3.2 pattern_observations

```
create table ai.pattern_observations (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  child_id uuid not null references child_record.children(id),  
  source_checkin_id uuid references child_record.child_checkins(id),  
  observation_type text not null,  
  extracted_value jsonb not null,  
  confidence_score numeric,  
  risk_level text not null,  
  model_name text not null,  
  model_version text not null,  
  prompt_version text not null,  
  created_at timestamptz not null default now()  
);
```

3.3 behaviour_patterns

```
create table ai.behaviour_patterns (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  child_id uuid not null references child_record.children(id),  
  pattern_type text not null,  
  pattern_summary text not null,  
  evidence_window_start timestamptz not null,  
  evidence_window_end timestamptz not null,  
  evidence_count int not null,  
  confidence_score numeric,  
  risk_level text not null,  
  child_safe_summary text,  
  adult_summary text,  
  status text not null default 'active',  
  created_at timestamptz not null default now(),  
  reviewed_by_user_id uuid,  
  reviewed_at timestamptz  
);
```

3.4 pattern_evidence_links

```
create table ai.pattern_evidence_links (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  behaviour_pattern_id uuid not null references ai.behaviour_patterns(id),  
  source_type text not null,  
  source_id uuid not null,  
  contribution_reason text,  
  created_at timestamptz not null default now()  
);
```

3.5 child_output_policy_decisions

```

create table ai.child_output_policy_decisions (
  id uuid primary key,
  tenant_id uuid not null references platform.tenants(id),
  child_id uuid not null references child_record.children(id),
  ai_output_id uuid not null,
  decision text not null,
  decision_reason text not null,
  risk_level text not null,
  requires_adult_review boolean not null default false,
  created_at timestamptz not null default now()
);

```

3.6 ai_outputs

```

create table ai.ai_outputs (
  id uuid primary key,
  tenant_id uuid not null references platform.tenants(id),
  child_id uuid,
  ai_job_id uuid not null,
  output_type text not null,
  output_text text not null,
  confidence_score numeric,
  model_name text not null,
  model_version text not null,
  prompt_version text not null,
  audience text not null,
  created_at timestamptz not null default now(),
  reviewed_by_user_id uuid,
  reviewed_at timestamptz,
  review_status text not null default 'pending',
  review_notes text
);

```

3.7 audit_events

```

create table audit.audit_events (
  id uuid primary key,
  tenant_id uuid not null,
  actor_user_id uuid,
  subject_type text not null,
  subject_id uuid,
  action text not null,
  event_time timestamptz not null default now(),
  ip_address inet,
  user_agent text,
  lawful_basis text,
  reason text,
  metadata jsonb not null default '{}'::jsonb
);

create index audit_events_tenant_time_idx
on audit.audit_events (tenant_id, event_time desc);

```

3.8 Row-Level Security pattern

```

alter table child_record.child_checkins enable row level security;
alter table ai.pattern_observations enable row level security;
alter table ai.behaviour_patterns enable row level security;
alter table ai.ai_outputs enable row level security;

create policy tenant_isolation_child_checkins
on child_record.child_checkins
using (
    tenant_id = current_setting('app.current_tenant_id')::uuid
);

create policy tenant_isolation_pattern_observations
on ai.pattern_observations
using (
    tenant_id = current_setting('app.current_tenant_id')::uuid
);

create policy tenant_isolation_behaviour_patterns
on ai.behaviour_patterns
using (
    tenant_id = current_setting('app.current_tenant_id')::uuid
);

create policy tenant_isolation_ai_outputs
on ai.ai_outputs
using (
    tenant_id = current_setting('app.current_tenant_id')::uuid
);

```

3.9 API transaction tenant context

```

select set_config('app.current_tenant_id', :tenant_id, true);
select set_config('app.current_user_id', :user_id, true);
select set_config('app.current_role', :role, true);

```

4. Structured Extraction Example

4.1 Child input

Lunch was too loud and I hid in the toilet. I felt angry and shaky.

4.2 Extracted structure

```
{
  "environment": "school_lunch_hall",
  "sensory_factor": "noise",
  "emotion": ["anger", "anxiety"],
  "body_signal": ["shaky"],
  "coping_action": "hid_in_toilet",
  "support_needed": true,
  "risk_level": "medium",
  "confidence": 0.82
}
```

4.3 Pattern aggregation logic

```
IF sensory_factor = noise
AND environment = school_lunch_hall
AND distress_score >= medium
AND occurrences >= 3
WITHIN 14 days
THEN create pattern: "Lunch noise may be difficult"
```

4.4 Output policy

```
Low-risk pattern
-> child-facing reflection allowed

Medium-risk pattern
-> child-facing reflection allowed + adult review item

High-risk pattern
-> no direct AI interpretation to child
-> safeguarding workflow
-> trusted adult notification according to policy
```

5. PlantUML: Pattern Recognition Sequence

```

@startuml
actor Child
participant "Child App" as App
participant "API" as API
participant "Safety Pre-screen" as Safety
participant "Pattern AI" as AI
participant "Rules Engine" as Rules
participant "PostgreSQL" as DB
participant "Adult Review Queue" as Review

Child -> App: submits check-in or journal entry
App -> API: send child input
API -> DB: store raw child check-in
API -> Safety: pre-screen input

alt high-risk content detected
    Safety -> Rules: trigger safeguarding workflow
    Rules -> DB: create safeguarding event
    Rules -> Review: urgent adult/professional review
    API -> App: show calm support message
else low/medium risk
    Safety -> AI: extract structured observations
    AI -> DB: store pattern observations
    AI -> DB: update behaviour patterns
    AI -> Rules: classify output policy
    alt low-risk child reflection
        Rules -> DB: log policy decision
        API -> App: show child-safe reflection
    else medium-risk pattern
        Rules -> DB: log policy decision
        Rules -> Review: create adult review item
        API -> App: show gentle child-safe reflection
    end
end
end
@enduml

```

6. Decision Summary

- Use PostgreSQL first as the primary transactional system of record.
- Use pooled multi-tenancy for MVP, with tenant_id and PostgreSQL Row-Level Security.
- Support promotion to dedicated schema/database for enterprise and high-assurance customers.
- Store child-facing AI data as governed records with source evidence links, model version, prompt version, confidence, risk level, review status, and audit history.
- Show only child-safe, non-diagnostic reflections to children.
- Route high-risk content to safeguarding workflows rather than general AI conversation.